

SE4450 - Team Retrospective Team Ctrl + Incision

1. Process Overview

Our team, Ctrl + Incision, consisted of five student developers, each taking on cross-functional responsibilities while holding two defined leadership roles. Elise Drouillard served as Product Owner, responsible for maintaining the backlog and writing acceptance criteria, while Hannah Taylor acted as Scrum Master and facilitator. The remaining team members, Janki Patel, Delaney McLachlan, and Muneeb Asif, contributed as developers, each with a focus area aligned to their strengths: Janki on QA and modularity, Delaney on 3D imaging and data stream integration, and Muneeb on UI/UX. Faculty oversight was provided by course instructor Dr. Abdelkader Ouda and faculty advisor Dr. Elvis Chen of Robarts Research Institute.

While the team charter initially framed our workflow around Scrum, our actual process evolved into something more milestone-driven. Rather than organizing work into fixed sprints with formal ceremonies, we divided the project into logical phases and assigned tasks based on each member's skills and interests. Responsibility boundaries were established collaboratively. If a task aligned naturally with someone's background or role focus, they took ownership of it. This approach felt more practical given our academic timeline and the exploratory nature of the project, which involved significant research into 3D ultrasound reconstruction and system refactoring.

Our primary collaboration tools were GitHub for version control, issue tracking, and pull request reviews, and a shared iMessage group for day-to-day async communication. Significant decisions were documented in GitHub to maintain a clear record. We held lightweight check-in cadences a few times per week rather than formal daily standups, and met with Dr. Elvis Chen on a scheduled basis for technical mentorship. While our process was less structured than a textbook Scrum implementation, it allowed flexibility for a project where task scope and technical direction shifted as we learned more about the existing codebase.

2. Sprint / Iteration Review

Sprint 1: Foundations & Architecture

- **Planned:** Establish a reproducible development environment, refactor existing code into modules, and set up a local database schema.
- **Delivered:** Successfully modularized core components, achieved a 100% reproducible setup, and integrated a SQLite database for patient scans and saved configurations.
- **Notable Variances:** While the software foundation was solid, advisor feedback noted a lack of hardware testing (tracking systems, frame grabber) and a need for better visualization of the high-level architecture, including a unified interface/application that

integrates all functionalities, allowing them to be used and accessed from a central application.

Sprint 2: Integration & Reconstruction

- Planned: Enable real-time stylus tracking, integrate the NDI tracker and PlusToolkit, and develop the VTK-based 3D volume reconstruction pipeline.
- Delivered:
 - Pipeline Integration: Successfully integrated live ultrasound acquisition and 6-DOF tracking using PlusServer.
 - 3D Reconstruction: Implemented workflow using PlusToolkit to convert sequences into volumetric datasets, driven by Python-scripted pipelines.
 - Stylus Tracking: Achieved ArUco-based stylus tracking with 6-DOF pose estimation and visual overlays.
 - UI Enhancements: Improved the interface to allow the easy conversion of 2D frames into 3D volumes, and added visualization tools/settings that allow users to apply colour and scalar opacity presets (transfer functions) to the volumetric data, which enables them to highlight specific tissue densities while making irrelevant structures transparent.
- Notable Variances:
 - Lack of Pipeline Automation: Individual components function correctly when manually triggered in sequence; however, the development of a fully automated workflow to bridge these modules was not completed.
 - Validation: Automated metrics for reconstruction accuracy (e.g., spatial error) and automated axial/pivotal calibration were moved to the backlog.
 - Lack of Centralized Application: The system currently lacks a centralized application; instead, each component operates through a distinct interface without an integrated navigation framework between tools.

Sprint	Planned Deliverables	Actual Deliverables	Key Issues / Variances
1	Environment Setup, Modularization, Initial DB	100% Reproducible Setup, Refactored modules, Scan storage DB	Hardware remained untested; UI language was too technical.

2	Stylus Calibration, NDI/PlusToolkit Integration, VTK Volume Rendering	6-DOF Tracked US Pipeline, ArUco Stylus Tracking, 3D Reconstruction Workflow	High manual overhead in XML configuration; Automated validation of 3D reconstruction moved to backlog.
---	---	--	--

3. What Worked Well

Incremental and Staged Development

One thing that worked well was the team's use of incremental, staged development. Approaching the work in smaller stages made it possible to validate individual components before integrating them into a larger pipeline and made it easier to distribute tasks across the team. For example, calibration workflows, tracked acquisition, stylus tracking, and reconstruction could each be developed and tested more independently before being combined. This made debugging more manageable and reduced the risk of spending large amounts of time troubleshooting the entire system at once.

Leveraging Existing Tools

Another aspect that worked well was the decision to build around existing specialized tools and libraries rather than implementing key system components from scratch. The team successfully incorporated PLUS Toolkit for tracked acquisition and reconstruction, OpenCV and ArUco markers for stylus tracking and camera-based pose estimation, SQLite for data storage, and VTK-based tools for volume rendering and interaction. This allowed the team to focus more on system integration, workflow design, and usability instead of rebuilding established low-level functionality. Given the complexity of the medical imaging and tracking domain, and the team's learning curve with the necessary tools and techniques, this was an effective way to keep the project feasible within the capstone timeline.

Prioritizing Core Functionality

Another strong aspect was the team's ability to stay focused on the core functionality of the project despite a growing number of possible extensions and ideal features. As development progressed, additional ideas, improvements, and "nice-to-have" capabilities were frequently identified through advisor feedback and ongoing experimentation. While many of these ideas were valuable, it became increasingly important to distinguish between what was essential for a working prototype and what could reasonably be deferred. The team made a conscious effort to prioritize the core technical goals of the project, including tracked acquisition, calibration, reconstruction, and visualization, rather than overextending effort across too many partially

implemented features. This helped ensure that the final deliverable was a functional and demonstrable prototype rather than a broader but less complete system.

4. What Didn't Work / Challenges

Challenge 1: Domain Complexity and Technical Steepness

The initial phase of the project was marked by a learning curve regarding medical imaging physics and spatial mathematics. Specifically, the team struggled to internalize the 3D transformation chains required to move data from image space to probe space, and finally to world space.

- **Root-Cause Analysis:** The technical barrier was large: a lack of prior experience with ultrasound-specific artifacts and the geometric complexity of 6-DOF (degrees of freedom) tracking. Without a clear end result mapped out initially, the team spent a lot of time in the "theory" phase before moving to a functional prototype.
- **Resolution:** This was mitigated by prioritizing the reconstruction module in Sprint 2, which forced a practical application of these mathematical foundations.

Challenge 2: Hardware Interfacing and Configuration Management

A primary technical failure occurred when attempting to interface the software with the ultrasound frame grabber and the NDI Aurora tracker. Configuring the PlusToolkit XML files to handle and synchronize these separate inputs proved to be a major bottleneck.

- **Root-Cause Analysis:** The PlusServer configuration requires high-precision port mapping and coordinate system definitions. Small mismatches in the XML configuration (such as orientation flags or path errors) resulted in "silent failures" where the system would run but produce spatially inaccurate data.
- **Resolution:** The team had to complete many time consuming trial-and-error attempts in order to get the hardware properly connected to our application.

Challenge 3: Synchronization of Team Knowledge and Advisor Alignment

A big operational challenge was the inconsistent attendance at weekly meetings with the project advisor. These sessions were intended for important progress updates and troubleshooting, but frequent absences led to a fragmented understanding of the project's progress and direction.

- **Root-Cause Analysis:** Missed meetings created an information asymmetry, requiring additional time to align the group. Additionally, team members were left in the dark regarding the status of interdependent tasks, which prevented cohesive progress. This led to a heavy software implementation load later in the project.
- **Impact:** This lack of synchronization was a primary reason for the advisor's request for more frequent meetings to manage technical debt and ensure milestones were met.

Challenge 4: Integration Friction and Non-Continuous Development

The team encountered "integration debt" where code additions were frequently built on outdated versions of the repository rather than the current refactored architecture.

- Root-Cause Analysis: Instead of a Continuous Integration (CI) mindset, many contributions were treated as separate pieces rather than iterative improvements to the central codebase. This made it difficult to merge independent contributions.
- Resolution: This forced members to perform additional work to merge these separate pieces into the main application dashboard.

5. Lessons Learned & Changes We Would Make

The first significant challenge was the inclusion of the ultrasound sector boundary in the reconstructed volume. Each raw ultrasound frame contains a fan-shaped active imaging region surrounded by black, non-diagnostic space. Because the pipeline did not include a preprocessing step to crop or mask frames to the active sector, the reconstruction algorithm treated the entire frame including the boundary and dead space as valid data. The root cause was an oversight in the frame processing stage of the pipeline design. The result was volumes with visible sector-shaped artifacts layered throughout, reducing the clarity and diagnostic usefulness of the reconstruction.

The second challenge involved scanning speed and its effect on reconstruction density. During testing, the operator moved the ultrasound probe across the scanning surface faster than the system's frame capture rate could accommodate. This produced spatial gaps between consecutively acquired frames that were too large for the reconstruction algorithm to interpolate across, resulting in visible holes and discontinuities in the 3D volume. The operator was essentially scanning blind with respect to density, relying on intuition rather than system guidance. This challenge highlighted that even a technically complete pipeline can produce poor results when the human-system interaction is not accounted for in the design.

The third challenge was the introduction of floating artifacts caused by lifting the probe during a scan. When the operator momentarily lost contact with the scanning surface or tilted the probe away, the Aurora NDI tracker continued reporting the probe's spatial position and the framegrabber continued capturing frames. These frames, now representing empty space or off-axis acoustic data, were reconstructed at their tracked positions, producing disconnected clusters of voxel data floating in space around the main volume. The root cause was the absence of any contact validation or frame quality gate in the acquisition pipeline. The system assumed that every tracked frame was valid and should be included in the reconstruction, with no mechanism to detect that the probe had lost surface contact. In a clinical or even a controlled testing scenario, this kind of artifact could obscure or distort the anatomy of interest. Addressing this would require either a hardware-level contact sensor or a software-based approach.

6. Individual Reflections

Muneeb Asif

Elise Drouillard

Through the development of our surgical intervention system, I significantly expanded my technical expertise in medical imaging and hardware-software integration. Working with the NDI Aurora tracker and ultrasound frame grabbers taught me the complexities of synchronizing high-precision spatial data with live video streams. I focused on the volume rendering pipeline, where I learned to use VTK to implement scalar opacity and color presets that allow users to isolate specific anatomical densities. Furthermore, designing the UI required a shift in my approach to frontend development. I had to move beyond standard design patterns to create an interface that prioritizes clinical clarity and ease-of-use, refactoring math-heavy descriptions into medical terminology to ensure the tool is intuitive for a surgical environment.

On the professional side, this project pushed me to develop essential leadership and project management skills. I found it necessary to step into a coordination role to ensure our objectives remained clear, and focused on maintaining the alignment of our tasks, specifically by helping to bridge the gap between our high-level architecture and the individual code contributions from the team. This experience taught me that effective leadership in a technical project is less about authority and more about maintaining a shared understanding of the project goals.

Delaney McLachlan

Over the course of this project, I developed a comprehensive understanding of a full medical imaging pipeline, from low-level hardware configuration through to 3D visualization. This involved setting up and integrating the Aurora NDI tracking system, an ultrasound device, and a framegrabber, configuring each component to communicate reliably before any data could be acquired. From there, I worked with Plus Server to manage the raw data acquisition process, handling the conversion of sequential 2D ultrasound frames into .mha files and ultimately reconstructing them into 3D volumes. The final stage of the pipeline involved building a VTK-based display system to render those volumes in a meaningful way. Working end-to-end across hardware, middleware, and visualization gave me a much deeper appreciation for how tightly coupled each layer of a real-time medical imaging system is, and how a failure at any one stage propagates through the entire pipeline.

On the professional side, this project pushed me to communicate in a more formal and technically precise way than I was accustomed to. Whether writing documentation, contributing to team discussions, or presenting progress to our faculty advisor, I became more deliberate

about using accurate terminology and structuring my explanations clearly for a technical audience. This is a skill I expect to carry directly into my engineering career.

Janki Patel

Throughout this project, I developed a stronger technical understanding of how interface design, workflow structure, and code organization affect the usability and maintainability of a larger software system. My work involved improving how users move through different parts of the application, displaying retrieved data, and refactoring some of the existing code given initially into a cleaner and more modular structure. Working with pre-existing functionality showed me how important it is to organize code in a way that is easier to navigate, update, and integrate with new features. This helped me better understand how backend structure and front-end workflow are closely connected, and how technical design decisions can directly affect both development efficiency and user experience.

On the professional side, this project helped me improve my communication and ability to work through ambiguity in a collaborative development environment. As different parts of the system were being developed in parallel, it was sometimes difficult to maintain a shared understanding of how individual components were supposed to connect and function together. I became more aware of how communication gaps, unclear assumptions, or incomplete context could slow integration and create confusion later in development. This experience taught me the importance of asking clearer questions, and communicating ideas in a way that supports both coordination and implementation.

Hannah Taylor

Throughout the development of this project, I developed a better understanding of how different components of a system interact, particularly in the context of data flow between modules. My work was primarily focused on the database and supporting infrastructure, where I helped organize and manage how data such as scan information and configuration parameters were stored and accessed. This involved designing a structure to support saving and retrieving data within the application, as well as ensuring that it integrated properly with other components of the system. This also gave me exposure to other parts of the system, such as working with the frame capture and acquisition setup, where I assisted in initializing and running components of the pipeline. Overall, this experience helped me understand how backend components support larger system functionality and how design decisions at this level can impact usability and integration.

On the professional side, this project helped me improve my communication and collaboration skills within a team working on a technically complex system. As the project progressed and our workflow became less structured, I became more aware of how a misalignment of expectations can affect integration and overall progress. I learned the importance of clear communication to keep everyone on the same page even in a more flexible

development environment. Overall, this experience reinforced the value of adaptability and coordination in collaborative software development.

7. Action Items for Future Projects

Based on the challenges encountered throughout this project, several actionable improvements have been identified for future team-based development efforts.

First, in future projects the team should prioritize early end-to-end system integration rather than developing components in isolation. A challenge in this project was integration friction caused by independently developed modules that were not continuously tested together. Establishing a working pipeline early, even in a minimal form, would help identify compatibility issues sooner and reduce integration overhead later in the project lifecycle.

Second, future projects should enforce more structured communication and knowledge-sharing practices, particularly for meetings with advisors. In this project, inconsistent attendance led to gaps in understanding and misalignment across team members. Ensuring consistent participation or documenting key takeaways from meetings and sharing them in a timely manner would help maintain a shared understanding of project direction and reduce redundancy in problem-solving efforts.

Third, hardware integration and configuration should be addressed earlier in the development process. The complexity of setting up and debugging hardware components such as the NDI tracker and frame grabber created delays later in the project. Allocating dedicated time early on for hardware validation would allow software development to proceed with more reliable assumptions about data flow and system behaviour.

Finally, future projects should place greater emphasis on defining clear intermediate milestones tied to functional system outcomes rather than isolated features. In this project, progress was sometimes measured by the completion of individual components rather than the successful operation of the overall pipeline. Structuring milestones around end-to-end functionality would encourage earlier validation of system behaviour and help ensure that all components contribute effectively to the final product.

Collectively, these action items for future projects aim to improve team coordination, reduce integration challenges, and enhance the overall robustness of future projects.